

Sprint 1

During the Sprint 1 of the Stokvel Management Platform, I worked mainly on implementing the contributions feature, specifically the contributions page and its functionality using HTML and JavaScript. This included displaying a user's payment history, filtering contributions, and calculating totals dynamically. This was my first time working on a group project using the scrum methodology and GitHub, and it was quite different from working individually. One of the main challenges I faced was understanding how to collaborate effectively using Git and GitHub. At first, I struggled with branching, committing correctly, and linking my work to Taiga task like TG-14. I also found it difficult to integrate my frontend work with other part of the system, especially when dealing with asynchronous data and ensuring everything worked together smoothly. Another challenge was following the Test-Driven approach, since I am more used to coding first then testing later. However, through daily standups and team discussions, I gradually became more comfortable with the workflow. I learned the importance of clear communication, small frequent commits, and writing meaningful commit messages. I also improved my understanding of modular JavaScript and how frontend components interact with backend data, even when using mock data. If I were to do this sprint again, I would start earlier with understanding the project structure and tools, especially Git and CI/CD, to avoid confusion later. I would also ask for help sooner when stuck instead of spreading too much time trying to figure things out alone. Overall, this sprint helped me gain practical experience in teamwork, version control, and Agile development, and I now feel more confident contributing to future sprints.

Sprint 2

During the Sprint 2 of the stokvel Management Platform, my main focus shifted from just implementing features to also thinking about the overall system design and architecture, including working with UML diagrams and improving what we previously built. Compared to sprint 1, this sprint required more planning and understanding of how different parts of the system connect, not just coding individual features. One of my biggest challenges in this sprint was time. Sprint 2 had a short duration period compared to sprint 1, and the workload increased since we had to come up with six new user stories for this sprint. My team and I not only had to continue implementing features, but also complete UML diagrams for the system architecture, prepare for the sprint review, and refine previous work. Because of this, I felt more pressure and had to rush certain tasks, which affected the quality and depth of some parts of my work. Another challenge was dealing with the backlog of incomplete user stories from Sprint 1. Some features, like fully integrating contributions with backend logic and improving validation, were not fully completed earlier and had to be carried over. At the same time, Sprint 2 introduced

new user stories, such as treasurer dashboard improvements and architectural documentation, which made it difficult to balance between finishing old tasks and starting new ones. I also found the UML and architecture part challenging. Understanding how to correctly represent the system using diagrams like use case, sequence, and component diagrams took time, especially making sure they matched the actual implementation. However, this helped me better understand how the frontend and backend should interact, which I didn't fully consider in Sprint 1. On the positive side, I improved in a few areas. I became more comfortable using GitHub for version control, and I was more consistent with commits compared to Sprint 1. I also had a better understanding of how to structure my code and think ahead about how features will integrate into the full system. Working with mock data also helped me simulate real system behavior more effectively. If I were to improve, I would manage my time better by prioritizing backlog items earlier instead of focusing too much on new features. I would also start the UML diagrams earlier, since they require more thinking than coding. Additionally, I would communicate more with teammates when tasks start overlapping, to avoid duplication or confusion. Overall, Sprint 2 was more challenging than Sprint 1 due to the shorter timeframe and increased complexity, but it helped me grow in understanding software architecture, planning, and balancing multiple responsibilities within a sprint.

Sprint 3

During the Sprint 3 of the stokvel Management Platform, my main focus shifted from just implementing features to also thinking about the overall system design and architecture, including working with UML diagrams and improving what we previously built. Compared to sprint 2, this sprint required more planning and understanding of how different parts of the system connect, not just coding individual features. One of my biggest challenges in this sprint was time. Sprint 3 had a short duration period compared to sprint 2, and the workload increased since we had to come up with six new user stories for this sprint. My team and I not only had to continue implementing features, but also complete UML diagrams for the system architecture, prepare for the sprint review, and refine previous work. Because of this, I felt more pressure and had to rush certain tasks, which affected the quality and depth of some parts of my work. Another challenge was dealing with the backlog of incomplete user stories from Sprint 2. Some features, like fully integrating contributions with backend logic and improving validation, were not fully completed earlier and had to be carried over. At the same time, Sprint 3 introduced new user stories, such as treasurer dashboard improvements and architectural documentation, which made it difficult to balance between finishing old tasks and starting new ones. I also found the UML and architecture part challenging. Understanding how to correctly represent the system using diagrams like use case,

sequence, and component diagrams took time, especially making sure they matched the actual implementation. However, this helped me better understand how the frontend and backend should interact, which I didn't fully consider in Sprint 2. On the positive side, I improved in a few areas. I became more comfortable using GitHub for version control, and I was more consistent with commits compared to Sprint 2. I also had a better understanding of how to structure my code and think ahead about how features will integrate into the full system. Working with mock data also helped me simulate real system behavior more effectively. If I were to improve, I would manage my time better by prioritizing backlog items earlier instead of focusing too much on new features. I would also start the UML diagrams earlier, since they require more thinking than coding. Additionally, I would communicate more with teammates when tasks start overlapping, to avoid duplication or confusion. Overall, Sprint 3 was more challenging than Sprint 2 due to the shorter timeframe and increased complexity, but it helped me grow in understanding software architecture, planning, and balancing multiple responsibilities within a sprint.

Sprint 4

During Sprint 4 of the Stokvel Management Platform, my role became more focused on improving functionality, refining existing features, and ensuring the system was more user-friendly and reliable. Compared to Sprint 3, this sprint required more attention to integration and testing because most of the core features had already been developed. Instead of only building new functionality, we had to improve the quality of the platform, fix issues from previous sprints, and prepare the system for a more complete demonstration. One of the main challenges I faced during Sprint 4 was balancing feature refinement with debugging and testing. As the project became larger, small changes in one part of the system sometimes affected other components unexpectedly. This meant I had to spend more time testing features thoroughly to ensure they worked correctly with the rest of the application. Since multiple team members were working on connected parts of the platform simultaneously, it was sometimes difficult to coordinate updates and avoid merge conflicts. Another challenge was managing the workload alongside the sprint deadline. Sprint 4 required us to not only complete our assigned tasks but also improve the overall user experience and fix issues identified during reviews from earlier sprints. Because of the limited time, prioritising tasks became very important, especially when deciding between implementing additional improvements and ensuring existing features were stable. My main contribution during Sprint 4 was continuing work on the payouts functionality by refining the `payouts.js` and `payouts.html` files and improving the disbursement process within the platform. I worked on making the disbursement button more functional and user-friendly, ensuring that payouts could be processed more smoothly and clearly for users. This task improved my

understanding of frontend interaction, event handling in JavaScript, and how user actions connect with the system workflow. During this sprint, I also improved my collaboration and development workflow skills. I became more confident using GitHub for version control, resolving merge conflicts, and keeping my commits organised. I also learned the importance of testing continuously throughout development instead of waiting until the end of the sprint. Working on integration issues helped me better understand how different parts of the frontend and backend interact within a full-stack system. On the positive side, Sprint 4 helped me improve my problem-solving skills and ability to work under pressure. Compared to earlier sprints, I felt more confident contributing to discussions about system improvements and debugging issues. I also became better at analysing problems logically before making changes to the code, which reduced unnecessary mistakes. If I were to improve in future projects, I would focus more on starting testing earlier and communicating more frequently with teammates about updates that may affect shared files. I would also spend more time planning before implementation so that fewer changes would need to be made later in the sprint. Additionally, I would improve documentation throughout development rather than leaving most of it until the end. Overall, Sprint 4 was challenging because it focused heavily on integration, debugging, and refinement rather than just implementing features. However, it helped me gain a better understanding of software testing, collaboration, version control, and the importance of maintaining stable and user-friendly functionality within a team project.